

Class Activity

Spring Boot Configuration Challenge

Objective

In this activity, you will practice two Spring Boot configuration techniques covered in the course notes:

- creating a REST endpoint that reads values from `application.properties` using `@Value`; and
- changing the embedded server port using `application.properties`.

The goal is to reinforce the idea that application behavior and infrastructure settings can be changed through configuration rather than by hard-coding values directly in Java code. The PDF shows that `/teaminfo` should retrieve its values from `application.properties`, and that changing `server.port` changes where the embedded server listens.

Scenario

You are developing a small Spring Boot service for a **college robotics club**.

The club wants a simple endpoint that displays information about the current robotics team. The team name and faculty advisor may change from semester to semester, so these values should **not be hard-coded directly inside the controller**.

The club also wants the application to run on a port different from Spring Boot's default 8080, because another application is already using that port.

Your task is to configure and test the application accordingly.

Task 1 — Add Team Properties

Open:

```
src/main/resources/application.properties
```

Add two custom properties:

```
robotics.team.name=Titan Robotics
```

```
robotics.advisor.name=Dr. Morgan
```

You may use different values if you prefer.

Task 2 — Inject the Properties

In your REST controller, create fields for the team name and advisor name and inject the property values using `@Value`.

For example:

```
@Value("${robotics.team.name}")  
  
private String teamName;
```

```
@Value("${robotics.advisor.name}")  
  
private String advisorName;
```

The PDF explains this same configuration-injection process using properties stored in `application.properties` and injected into a controller with `@Value`.

Task 3 — Create a `/teaminfo` Endpoint

Create a new endpoint:

```
@GetMapping("/teaminfo")
```

The endpoint should return both injected values.

For example:

```
@GetMapping("/teaminfo")  
  
public String getTeamInfo() {  
    return "Team: " + teamName +  
        ", Advisor: " + advisorName;  
}
```

Expected output:

```
Team: Titan Robotics, Advisor: Dr. Morgan
```

At this stage, if you have not yet changed the server port, test the endpoint at:

```
http://localhost:8080/teaminfo
```

The `/teaminfo` lab in the PDF uses the same two-step process: define properties first, then inject them into the Java class using `@Value`.

Task 4 — Change the Server Port

Now modify the application's embedded-server configuration.

In `application.properties`, add:

```
server.port=9090
```

Restart the application.

Check the console and confirm that the embedded server is now running on port:

```
9090
```

The notes demonstrate the same technique by changing Spring Boot's usual port from 8080 to another value using the `server.port` property.

Task 5 — Test the New URL

Try the old URL:

```
http://localhost:8080/teaminfo
```

It should no longer reach this Spring Boot application.

Now access:

```
http://localhost:9090/teaminfo
```

You should see your robotics team information.

Task 6 — Make a Configuration Change Without Changing Java Code

Change only the following values in `application.properties`:

```
robotics.team.name=Quantum Robotics
```

```
robotics.advisor.name=Prof. Chen
```

Do **not** modify the Java controller.

Restart or allow DevTools to restart the application, then refresh:

`http://localhost:9090/teaminfo`

The browser should now display the new values.

This demonstrates the main benefit of externalized configuration: the Java logic can remain unchanged while configurable values are modified separately. The PDF identifies `application.properties` as Spring Boot's standard location for this kind of configuration.

What to Show for the Grade

You do not need to submit a written answer to theory questions. Complete the coding activity and **show the working application to the professor.**

Be ready to demonstrate:

1. your `application.properties` file with:

```
robotics.team.name=...  
robotics.advisor.name=...  
server.port=9090
```

2. the `@Value` fields in your controller;
3. the `/teaminfo @GetMapping`;
4. the console showing that Spring Boot is running on port 9090;
5. the working browser output at:

`http://localhost:9090/teaminfo`

6. that changing the team properties changes the browser output **without changing the controller code.**

Challenge

If you finish early, add one more custom property:

```
robotics.season=Fall 2026
```

Inject it using `@Value` and include it in the `/teaminfo` response.